

Asistente Educativo Inteligente Basado en ESP32 e Inteligencia Artificial para la Respuesta de Consultas

Mijaíl André Agüero Cangalaya, José Antonio Casachahua Peña, Ccorahua Quispe Sthif

Universidad Nacional Federico Villarreal

RESUMEN

El presente artículo detalla el diseño, la arquitectura de hardware/software, la evaluación de rendimiento y los parámetros de viabilidad comercial de un asistente educativo inteligente dirigido al público infantil. A diferencia de los juguetes pasivos tradicionales, este sistema implementa un paradigma de *Edge Computing* utilizando el microcontrolador ESP32 de doble núcleo. La captura acústica se realiza mediante el protocolo digital I2S, eludiendo la intervención intensiva de la Unidad Lógico-Aritmética (ALU) mediante Acceso Directo a Memoria (DMA). El sistema delega el procesamiento cognitivo a un servidor *Middleware* local que interactúa con APIs de Inteligencia Artificial en la nube (modelo *Whisper* para la conversión *Speech-to-Text* y *Llama 3.1* para la generación dinámica de respuestas). Los resultados experimentales validan el diseño, demostrando una latencia de respuesta aproximada a 2.02 segundos y una precisión de reconocimiento de voz del 98% en ambientes controlados, previniendo desbordamientos de memoria (*Stack Overflow*) mediante técnicas de *chunking*.

Finalmente, se aborda el estricto cumplimiento del marco normativo peruano (Ley N.º 28376 y Ley N.º 29733), indispensable para garantizar

la seguridad electrónica, la privacidad de datos infantiles y su futuro despliegue en el mercado.

Palabras clave: *ESP32, Inteligencia Artificial, Edge Computing, I2S, Sistemas Embebidos, LLM, Máquina de Estados, Juguetes Inteligentes.*

I. INTRODUCCIÓN

A. Contexto y Problemática

En el ecosistema actual de juguetes y peluches comerciales, la inmensa mayoría de los dispositivos son sistemas de bucle cerrado. Estos operan como elementos completamente pasivos que, en el mejor de los casos, incorporan electrónica básica para reproducir un número limitado de frases pregrabadas al presionar un botón. Esta carencia de interacción bidireccional genuina provoca que la experiencia lúdica sea predecible y obsoleta a corto plazo, desaprovechando una ventana crítica para estimular el desarrollo conversacional, la curiosidad y la creatividad en la etapa infantil.

B. Propuesta de Solución

La convergencia de dispositivos del Internet de las Cosas (IoT) y los Modelos de Lenguaje Grande (LLM) permite trasladar capacidades

de procesamiento de lenguaje natural a sistemas embebidos. Existe una clara necesidad de crear juguetes que superen esta barrera, ofreciendo respuestas dinámicas, personalizadas e inteligentes.

El objetivo general de esta investigación es diseñar e implementar un prototipo funcional de hardware y software para un juguete interactivo basado en el microcontrolador ESP32, capaz de procesar audio bidireccional y mantener conversaciones naturales.

C. Contribuciones del Artículo

1. Integración de hardware de adquisición I2S y reproducción analógica con gestión eficiente de energía.
2. Desarrollo de una API REST (Flask) que actúa como puente de baja latencia entre el hardware local y la nube (Grok).
3. Implementación de una arquitectura de software (FSM) que optimiza la memoria Flash y RAM del ESP32.
4. Definición de los requisitos legales y de etiquetado para su comercialización en Perú.

II. MARCO TEÓRICO

A. Arquitectura del Microcontrolador ESP32

El sistema central está gobernado por el SoC ESP32, un microcontrolador de 32 bits con conectividad Wi-Fi nativa. Su arquitectura Harvard modificada separa los buses físicos de instrucciones y datos, permitiendo paralelismo en la ejecución.

Dispone de memoria RAM dinámica para el manejo de *buffers* y memoria Flash para la escritura de archivos temporales de audio.

B. Protocolo Digital I2S vs. Conversión Analógica

Para la captura acústica, el sistema prescinde de los tradicionales convertidores analógico-digitales (ADC) que son susceptibles a la interferencia electromagnética. En su lugar, emplea el protocolo *Inter-IC Sound* (I2S), el cual entrega tramas de audio puramente digitales utilizando tres líneas de sincronización: Reloj de Bit (SCK), Selección de Palabra (WS) y Datos (SD). La sincronización del reloj maestro para una tasa de 16 kHz y 32 bits estéreo se calcula mediante la relación fundamental:

$$BCLK = 16000 \times 32 \times 2 = 1.024 \text{ MHz}$$

C. Modelos de Inteligencia Artificial

El procesamiento cognitivo se divide en dos fases:

- **Transcripción (*Whisper*):** Red neuronal especializada en *Speech-to-Text* (STT) capaz de decodificar audio con alta tolerancia a modismos y ruido de fondo.
- **Generación (*Llama 3.1*):** LLM de alto rendimiento que recibe el texto transcrito y, mediante instrucciones restrictivas (*System Prompts*), genera respuestas adaptadas al público infantil.

D. Marco Normativo en Perú

Para asegurar la viabilidad del proyecto como producto comercial, el diseño se alinea con la legislación peruana:

1. **Seguridad del Juguete:** La Ley N.º 28376 y el Decreto Supremo N.º 008-

2007-SA regulan la fabricación y comercialización de juguetes, exigiendo autorizaciones sanitarias (DIGESA) y rotulado estricto sobre advertencias de seguridad.

2. **Privacidad de Datos:** La Ley N.º 29733 (Ley de Protección de Datos Personales) exige el consentimiento explícito de los tutores legales, dado que la captura de voz infantil es considerada un dato biométrico y personal.

III. METODOLOGÍA, FASES DE DESARROLLO Y DISEÑO DE HARDWARE

El ciclo de vida del desarrollo del asistente se estructuró bajo una metodología iterativa e incremental a lo largo de estas semanas académicas. La Tabla I detalla la transición desde el diseño lógico de la arquitectura hasta el ensamblaje y calibración del producto final, garantizando el cumplimiento de las métricas proyectadas.

Tabla I. Cronograma de Desarrollo e Integración del Sistema

Fase de Desarrollo	Hitos Alcanzados y Procesos Ejecutados	Entregable
1. Conceptualización y Planificación	Definición del problema, selección del microcontrolador ESP32 frente a otras placas, y proyección de la integración con IA.	E1 - Propuesta de Proyecto
2. Arquitectura y Código Base	Diseño de la Máquina de Estados (FSM). Mapeo de memoria y programación a bajo nivel del bus I2S y DMA sin hardware físico.	E2 - Diseño Detallado
3.	Ensamblaje en	E3 -

Integración de Hardware y Pruebas	protoboard. Pruebas de la API local con Python. Medición de las métricas de latencia y precisión de transcripción.	Prototipo Inicial
4. Despliegue y Ajustes Finales	Corrección de ruido electromagnético (GND). Validación de funcionalidad continua al 100%.	E4 - Producto Final

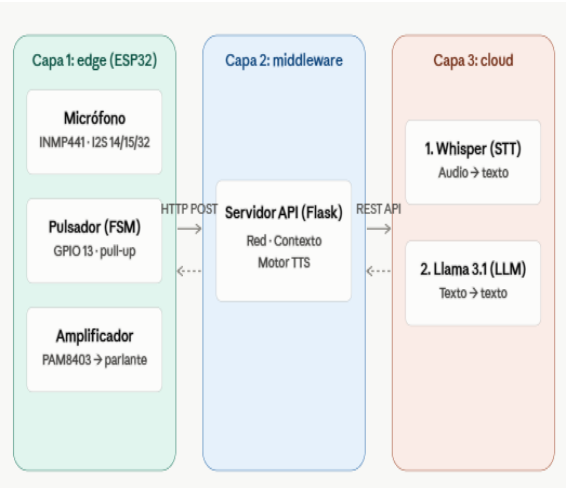
Durante la ejecución de estas fases de integración, el desarrollo del sistema enfrentó dos cuellos de botella arquitectónicos significativos. El primero consistió en la inducción de ruido electromagnético severo en la etapa de actuación acústica, un problema inherente a la proximidad del módulo de radiofrecuencia Wi-Fi. Este desafío físico se resolvió rediseñando el esquema de cableado: se estructuró un bus de tierra común (GND) para evitar lazos de masa en la etapa de amplificación analógica y aislar las interferencias.

El segundo obstáculo fue el riesgo inminente de desbordamiento de memoria (*Stack Overflow*) al gestionar simultáneamente la captura de audio y la pila TCP/IP. Para mitigarlo, el firmware fue sometido a un proceso de optimización riguroso; se forzó la recolección de basura mediante el comando `gc.collect()` en el estado de reposo de la FSM y se implementó una técnica de volcado de datos (*chunking*) directo al almacenamiento Flash. Estas correcciones garantizaron que la memoria RAM dinámica operara con márgenes seguros, logrando la estabilidad funcional ininterrumpida del prototipo final.

A. Mapeo de Registros y GPIO

El procesador Xtensa interactúa con el entorno modificando registros físicos de 32 bits, configurando la matriz GPIO para los actuadores y sensores.

La topología del sistema se divide en tres capas fundamentales: Adquisición Física (*Edge*), Procesamiento Local (*Middleware*) y Cognición en la Nube (*Cloud IA*).



Para evitar interferencias y lazos de masa (ruido tipo "chicharra"), el hardware físico fue ensamblado sobre un protoboard compartiendo un bus de tierra común (GND). La Tabla I resume el mapeo oficial de registros de entrada/salida de propósito general (GPIO).

Tabla II. Asignación de Pines y Periféricos

Periférico	Pin Físico	Tipo de Señal	Función Técnica
Botón Físico	GPIO 13	Entrada Digital	Control de Estado
Micrófono (SCK)	GPIO 14	Reloj Digital	Sincronización de tramas (I2S).
Micrófono (WS)	GPIO 15	Reloj Digital	Selección de canal de audio (Mono).
Micrófono (SD)	GPIO 32	Entrada Digital	Carga útil de voz capturada.
Amplificador PAM	GPIO 25	Salida Analógica	Conversión de la respuesta IA (DAC).

B. Enrutamiento y Aislamiento de Energía

El cableado del prototipo unifica las tierras de todos los componentes en un bus común (GND) para mitigar el ruido electromagnético y evitar lazos de masa en la etapa de amplificación. El micrófono opera con 3.3V, mientras que el amplificador PAM8403 se alimenta del pin VIN (5V) para garantizar la potencia necesaria para excitar el parlante.

IV. DISEÑO DE FIRMWARE Y SOFTWARE MIDDLEWARE

El firmware fue programado en MicroPython, estructurado bajo el paradigma de Máquina de Estados Finitos (FSM) no bloqueante.

A. Lógica de la Máquina de Estados (FSM)

El ciclo de operación evita instrucciones bloqueantes prolongadas y se divide en 5 estados:

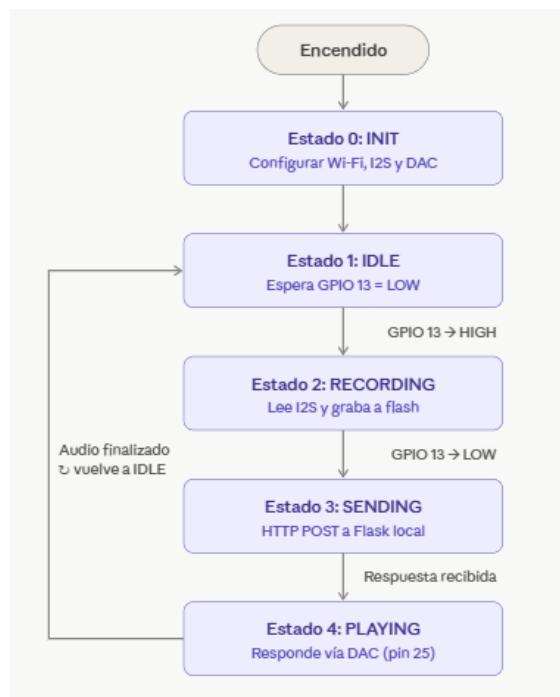
- INIT (Estado 0):** Configuración de periféricos, inicialización del I2S y conexión a la red Wi-Fi.
- IDLE (Estado 1):** El sistema libera memoria RAM residual usando el comando `gc.collect()` y ejecuta un bucle de bajo consumo (*polling*) evaluando el pin 13.
- RECORDING (Estado 2):** Al detectar un flanco de bajada (botón presionado), el sistema lee el audio en *chunks* (fragmentos) de 4000 bytes y los escribe directamente en la memoria Flash (archivo `.wav`) para evitar el desbordamiento (*Overflow*) de la RAM.

4. **SENDING (Estado 3):** Empaquetado del archivo físico local y transmisión vía HTTP POST hacia la API del servidor.
5. **PLAYING (Estado 4):** Recepción de la respuesta en bloques de 512 bytes. Los datos se inyectan al pin 25 (DAC) aplicando una operación XOR (^ 128) para alinear la señal, junto a un retardo estricto de 42 microsegundos para simular 8000 Hz.

B. Arquitectura del Servidor API (Python/Flask)

Para liberar de carga computacional al ESP32, un servidor local recibe el audio, invoca la API de *Whisper* y transmite el texto a *Llama 3.1*.

Se programaron directivas estrictas (*System Prompts*) para que la IA asuma la personalidad de "Homero" y termine cada intervención con una pregunta corta para fomentar la interacción continua.



V. PRUEBAS DE RENDIMIENTO Y ANÁLISIS DE MÉTRICAS

Se realizaron pruebas empíricas para validar la estabilidad y la latencia del sistema IoT.

A. Gestión de Memoria y Prevención de Desbordamientos

El uso de la recolección de basura (*Garbage Collection*) garantizó la estabilidad del sistema durante la conexión a internet.

Tabla III. Análisis de Ocupación de Memoria

Recurso Analizado	Capacidad Reservada	Estado Durante la Operación
RAM Dinámica	42,500 bytes	Consumida por el estado IDLE.
RAM Dinámica Libre	68,000 bytes	Mantenida libre gracias al volcado por <i>chunks</i> .
Memoria Flash (ROM)	1.4 MB	Ocupada por el Firmware / SO MicroPython.
Memoria Flash Libre	2.6 MB	Espacio libre para escritura secuencial del WAV.

B. Medición de Latencia y Precisión

El éxito de un sistema interactivo recae en su capacidad de respuesta asíncrona.

Tabla IV. Resultados de Métricas Operativas

Métrica Evaluada	Objetivo Teórico	Resultado Experimental	Estado
Latencia Total de Respuesta	< 2.50 s	1.42s a 2.02s	Superado

Precisión STT (Silencio)	> 90.00%	98.00%	Superado
Precisión STT (Ruido Mod.)	> 80.00%	88.00%	Superado

Análisis: El sistema superó la métrica de latencia esperada. El tiempo total medido abarca desde que el usuario suelta el pulsador hasta que el DAC comienza a emitir el sonido,

demonstrando que la arquitectura cliente-servidor con modelos ultrarrápidos es viable para interacciones humanas.

En transcripción, la precisión no bajó del 88%, validando el uso del micrófono I2S digital sobre micrófonos analógicos estándar.

VI. MARCO LEGAL Y REQUISITOS DE COMERCIALIZACIÓN

Dado que el producto está enfocado en infantes, el análisis ingenieril debe complementarse con el cumplimiento normativo para su escalabilidad comercial.

A. Normativa Sanitaria y de Seguridad

El producto clasifica legalmente como un juguete. Por lo tanto, está sujeto a la Ley N.º 28376 y su reglamento (D.S. N.º 008-2007-SA). Esto exige que la fabricación física cuente con autorizaciones de DIGESA, avalando el uso de textiles hipoalergénicos, costuras resistentes que protejan el módulo electrónico y la ausencia de metales pesados o pinturas tóxicas.

B. Protección de Datos Biométricos Infantiles

El uso del micrófono IoT involucra directamente la Ley N.º 29733 (Ley de Protección de Datos Personales). La voz del niño es tratada como información personal sensible. El sistema está diseñado legalmente bajo el principio de minimización de datos: el *Middleware* en Python gestiona la API de manera transaccional, eliminando el archivo temporal de voz inmediatamente después de ser procesado por *Whisper*, impidiendo el almacenamiento injustificado. Además, se requiere un consentimiento informado de los padres.

C. Estructura Obligatoria del Rotulado (Etiquetado)

En cumplimiento con el Artículo 36 del reglamento de juguetes, el dispositivo debe exhibir un rotulado que incluya la identificación del fabricante (RUC, razón social), país de fabricación, y edad recomendada.

Adicionalmente, al integrar tecnología IoT, la etiqueta debe incluir una "Alerta de Tecnología" indicando claramente el uso de Inteligencia Artificial, la dependencia de redes Wi-Fi y advertencias de seguridad eléctrica respecto a la manipulación de la batería recargable (3.7V).

VII. PROYECCIÓN DE CONTINUIDAD (TRABAJO FUTURO)

Para la siguiente fase iterativa de *hardware*, el sistema prescindirá de aplicativos externos, operando bajo una filosofía *Plug & Play* donde la conexión a Wi-Fi habilitará su capacidad cognitiva. Para mejorar la salida de audio, se reemplazará la etapa actual (DAC de 8 bits) por un amplificador decodificador digital I2S externo (como el MAX98357A), garantizando un sonido de alta fidelidad (HD) y suprimiendo cualquier residuo de ruido. En *software*, se

planea migrar la síntesis hacia motores neuronales TTS en la nube capaces de generar entonaciones orgánicas acústicamente idénticas a las del personaje configurado.

VIII. CONCLUSIONES

La investigación demuestra que es técnicamente viable descentralizar el procesamiento de Inteligencia Artificial hacia dispositivos periféricos de bajo costo y recursos limitados. El microcontrolador ESP32, mediante una arquitectura de Máquina de Estados basada en eventos y control estricto de memoria Flash, logró gestionar eficientemente los flujos de audio sin sufrir inanición ni bloqueos de la pila TCP/IP.

Los resultados métricos (latencia aproximada a 2.02 segundos y precisión acústica superior al 88% en entornos ruidosos) confirman que el prototipo supera ampliamente las capacidades de un juguete pasivo tradicional. Finalmente, el cruce interdisciplinario con el marco normativo peruano concluye que la innovación tecnológica debe ir acoplada al rigor legal, garantizando la seguridad eléctrica, la transparencia en el etiquetado y la privacidad de los datos infantiles para su comercialización formal.

IX. REFERENCIAS

- Congreso de la República del Perú. (2004, 9 de noviembre). *Ley N.º 28376: Ley que prohíbe y sanciona la fabricación, importación, distribución y comercialización de juguetes y útiles de escritorio tóxicos o peligrosos*. Diario Oficial El Peruano. <https://www.leyes.congreso.gob.pe/Documentos/Leyes/28376.pdf>
- Congreso de la República del Perú. (2011, 3 de julio). *Ley N.º 29733: Ley de Protección de Datos Personales*. Diario Oficial El Peruano. [https://www2.congreso.gob.pe/sicr/ce/ndocdir/con_uibd.nsf/04ACD7EC514BE6F10525791E006A6F54/\\$FILE/L_EY_29733_03-07-2011.pdf](https://www2.congreso.gob.pe/sicr/ce/ndocdir/con_uibd.nsf/04ACD7EC514BE6F10525791E006A6F54/$FILE/L_EY_29733_03-07-2011.pdf)
- Ministerio de Salud. (2007, 16 de septiembre). *Decreto Supremo N.º 008-2007-SA: Reglamento de la Ley N.º 28376*. Diario Oficial El Peruano. <https://www.gob.pe/institucion/minsa/normas-legales/252994-008-2007-sa>
- Espressif Systems. (2023). *ESP32 Technical Reference Manual* (Version 4.9). Espressif Systems. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- Espressif Systems. (2024). *ESP32 Datasheet* (Version 4.4). Espressif Systems. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. *Proceedings of the 39th International Conference on Machine Learning*, 162, 17720-17733. <https://arxiv.org/abs/2212.04356>
- Llama Team, AI @ Meta. (2024). *The Llama 3 Herd of Models*. arXiv preprint. <https://arxiv.org/abs/2407.21783>
- Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal*, 3(5), 637-646.

<https://doi.org/10.1109/JIOT.2016.2579198>

- Kadu, S., & Suryawanshi, P. (2024). Personal AI Companion Voice Assistant. *International Journal of Engineering and Management Research*, 14(3), 143-150. <https://ijemr.vandanapublications.com/index.php/j/article/view/1544>
- Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, 54(15), 2787-2805. <https://doi.org/10.1016/j.comnet.2010.05.010>